

Notebook

February 13, 2025

1 Theoretische und praktische Anwendung der bedingten Wahrscheinlichkeiten

1.1 Aufgabe 4:

Der alleinstehende Bob hat zum Schutz vor Einbrechern eine Alarmanlage installiert. Er kann den Alarm durch seine Berufstätigkeit aber nicht hören und beauftragt seine Nachbarn John und Mary ihn zu informieren, wenn sie einen Alarm wahrnehmen. Nach mehreren Jahren modelliert Bob das Anrufverhalten seiner Nachbarn über bedingte Wahrscheinlichkeiten - Es gilt John = J, Mary = M, Alarm = Al:

$$\begin{array}{ll} P(J \mid Al) = 90\% & P(M \mid Al) = 70\% \\ P(J \mid \neg Al) = 5\% & P(M \mid \neg Al) = 1\% \end{array}$$

Mary ist schwerhörig und überhört den Alarm öfter als John. John verwechselt den Alarm von Bob's Haus mit dem Alarm von anderen Häusern. Der Alarm kann durch einen Einbruch, aber auch durch ein Erdbeben ausgelöst werden. Bob will aber nur von Alarmen bezüglich eines Einbruches informiert werden. Daraus ergeben sich die folgenden Zusammenhänge für die bedingten Wahrscheinlichkeiten:

- **Wahrscheinlichkeit, dass ein Alarm ausgelöst wurde, unter der Bedingung, dass sowohl ein Erdbeben eingetroffen ist, als auch ein Einbrecher in das Haus eingebrochen ist** $P(Al \mid Ein, Erd) = 95\%$
- **Wahrscheinlichkeit, dass ein Alarm ausgelöst wurde, unter der Bedingung, dass kein Erdbeben eingetroffen ist, aber ein Einbrecher in das Haus eingebrochen ist** $P(Al \mid Ein, \neg Erd) = 94\%$
- **Wahrscheinlichkeit, dass ein Alarm ausgelöst wurde, unter der Bedingung, dass ein Erdbeben eingetroffen ist, aber kein Einbrecher in das Haus eingebrochen ist** $P(Al \mid \neg Ein, Erd) = 29\%$
- **Wahrscheinlichkeit, dass ein Alarm ausgelöst wurde, unter der Bedingung, dass sowohl kein Erdbeben eingetroffen ist, als auch kein Einbrecher in das Haus eingebrochen ist** $P(Al \mid \neg Ein, \neg Erd) = 0.1\%$

Sowie gelten die Wahrscheinlichkeiten für einen Einbruch und eines Erdbebens mit:

$$P(Erd) = 0.2\% \quad P(Ein) = 0.1\%$$

1.1.1 Aufgaben):

Simulieren Sie das Beispiel mit einem Python Programm und berechnen Sie so empirisch die geforderten Wahrscheinlichkeiten.

1. Die Wahrscheinlichkeit, dass ein Alarm ausgelöst wird: $P(Al)$
2. Die Wahrscheinlichkeit, dass John anruft: $P(J)$
3. Die Wahrscheinlichkeit, dass Mary anruft: $P(M)$
4. Die Wahrscheinlichkeit, dass beide bei einem Einbruch anrufen: $P(J \wedge M \mid Ein)$
5. Die Wahrscheinlichkeit, dass beide bei keinem Einbruch anrufen: $P(J \wedge M \mid \neg Ein)$
6. Die Wahrscheinlichkeit, dass John und Mary anrufen: $P(J \wedge M)$
7. Die Wahrscheinlichkeit, dass ein Einbruch unter der Voraussetzung, dass John und Mary anrufen, stattfindet: $P(Ein \mid J \wedge M)$
8. Sind die Anrufe voneinander unabhängig

Demnach werden wieder die benötigten Bibliotheken `random` und `math` importiert.

```
[651]: import random
import math
```

Zuerst definieren wir die verschiedensten Wahrscheinlichkeiten

```
[652]: probability_john_if_alarm : float = 0.9
probability_mary_if_alarm : float = 0.7
probability_john_if_no_alarm : float = 0.05
probability_mary_if_no_alarm : float = 0.01
probability_earthquake : float = 0.002
probability_bulgary : float = 0.001
probability_alarm_if_bulgary_and_earthquake : float = 0.95
probability_alarm_if_bulgary_and_no_earthquake : float = 0.94
probability_alarm_if_no_bulgary_and_earthquake : float = 0.29
probability_alarm_if_no_bulgary_and_no_earthquake : float = 0.001
```

Danach werden die nötigen Variablen definiert, welche im Laufe der Simulation gezählt werden sollen.

```
[653]: count_alarm : int = 0
count_bulgary : int = 0
count_earthquake : int = 0
count_call_john : int = 0
count_call_mary : int = 0
count_call_both_if_bulgary : int = 0
count_call_both_if_no_bulgary : int = 0
count_call_both : int = 0
```

Demzufolge werden auch boolsche Variablen deklariert, welche den Ablauf der Simulation beschreiben. So sind Informationen wie z.B. `john_called` oder `mary_called` wichtig für den Simulationsverlauf.

```
[654]: alarm: bool
bulgary : bool
earthquake : bool
john_called : bool
mary_called : bool
```

Dann beginnt der Simulationsablauf. Zuvor werden aber noch zwei Funktionen generiert, welche den Simulationsablauf etwas übersichtlicher gestalten. Zum einen die Funktion `get_john_mary_called`, welche basierend auf den Alarmstatus: `True`, oder `False`, ausgibt, ob John, oder Mary anrufen. Dazu wird eine Zufallszahl für beide Fälle generiert, fällt diese unter den Schwellwert der bedingten Wahrscheinlichkeit des jeweiligen Falles, dass John oder Mary anrufen, werden die Variablen `john_called`, bzw. `mary_called` auf `True` gesetzt und zurückgegeben.

```
[655]: def get_john_mary_called(alarm : bool):
    global john_called
    global mary_called

    global probability_john_if_alarm
    global probability_mary_if_alarm
    global probability_john_if_no_alarm
    global probability_mary_if_no_alarm

    generate_john_call = random.random()
    generate_mary_call = random.random()

    if alarm:
        if generate_john_call < probability_john_if_alarm:
            john_called = True

        if generate_mary_call < probability_mary_if_alarm:
            mary_called = True

    else:
        if generate_john_call < probability_john_if_no_alarm:
            john_called = True

        if generate_mary_call < probability_mary_if_no_alarm:
            mary_called = True

    return john_called, mary_called
```

Des Weiteren wird die Funktion `get_alarm_and_call_status` deklariert, welche basierend auf der bedingten Wahrscheinlichkeit des Alarms auf die einzelnen Fälle, den Alarmstatus generiert und ausgibt, sowie die in der Funktion davor generierten Anruferzustände von John und Mary.

```
[656]: def get_alarm_and_call_status(alarm_probability_case):
    global john_called
    global mary_called

    global probability_john_if_alarm
    global probability_mary_if_alarm
    global probability_john_if_no_alarm
    global probability_mary_if_no_alarm
```

```

generate_alarm = random.random()

if generate_alarm < alarm_probability_case:
    alarm = True

    john_called, mary_called = get_john_mary_called(alarm)

else:
    alarm = False

    john_called, mary_called = get_john_mary_called(alarm)

return alarm, john_called, mary_called

```

Nun beginnt die eigentliche Simulation. Diese läuft über 10000 Iterationen und generiert zunächst einen Einbruch und ein Erdbeben. Danach werden die vier Fälle aus der Aufgabenstellung betrachtet.

Wenn ein Einbruch und ein Erdbeben stattfindet, werden die Variablen `bulgary` und `earthquake` auf `True` gesetzt. Daraufhin wird durch die oben definierten Funktionen die Wahrscheinlichkeiten, basierend auf dem Fall, für den Alarm und das Anrufsverhalten für John und Mary berechnet. Dieser Zyklus wird für alle vier Fälle angewandt.

Danach wird für die Iteration überprüft, welche Situationen eingetroffen sind und der jeweilige Zähler um eins erhöht. Z.B. wird der Zähler `count_alarm` um eins erhöht, sollte ein Alarm im Laufe der Iteration ausgelöst worden sein.

Am Ende der Simulation werden dann die Anzahl aller eingetroffenen Zustände ausgegeben.

```

[657]: iterations : int = 100000
for _ in range(iterations):
    john_called, mary_called = False, False

    generate_bulgary = random.random()
    generate_earthquake = random.random()

    if generate_bulgary < probability_bulgary and generate_earthquake <
↳ probability_earthquake:
        bulgary, earthquake = True, True

        alarm, john_called, mary_called =
↳ get_alarm_and_call_status(probability_alarm_if_bulgary_and_earthquake)

    elif generate_bulgary < probability_bulgary and generate_earthquake >
↳ probability_earthquake:
        bulgary, earthquake = True, False

```

```

        alarm, john_called, mary_called =
↳get_alarm_and_call_status(probability_alarm_if_bulgary_and_no_earthquake)

        elif generate_bulgary > probability_bulgary and generate_earthquake <
↳probability_earthquake:
            bulgary, earthquake = False, True

        alarm, john_called, mary_called =
↳get_alarm_and_call_status(probability_alarm_if_no_bulgary_and_earthquake)

    else:
        bulgary, earthquake = False, False

        alarm, john_called, mary_called =
↳get_alarm_and_call_status(probability_alarm_if_no_bulgary_and_no_earthquake)

    if alarm:
        count_alarm += 1

    if bulgary:
        count_bulgary += 1

    if earthquake:
        count_earthquake += 1

    if john_called:
        count_call_john += 1

    if mary_called:
        count_call_mary += 1

    if john_called and mary_called:
        count_call_both += 1

    if john_called and mary_called and bulgary:
        count_call_both_if_bulgary += 1

    if john_called and mary_called and not bulgary:
        count_call_both_if_no_bulgary += 1

count_cases = ["Anzahl, wie oft die Simulation itertiert wurde", "Anzahl, wie
↳oft der Alarm ausgelöst wurde", "Anzahl, wie oft eingebrochen wurde",
                "Anzahl, wie oft ein Erdbeben stattfand", "Anzahl, wie oft John
↳angerufen hat", "Anzahl, wie oft Mary angerufen hat",

```

```

        "Anzahl, wie oft beide angerufen haben", "Anzahl, wie oft beide_
↪angerufen haben, als ein Einbruch stattfand", "Anzahl, wie oft beide_
↪angerufen haben, als kein Einbruch stattfand"]

count_list = [iterations, count_alarm, count_bulgary, count_earthquake,
↪count_call_john, count_call_mary, count_call_both,
               count_call_both_if_no_bulgary, count_call_both_if_no_bulgary]

for i in range(len(count_list)):
    print(f"Die {count_cases[i]} beträgt {count_list[i]}.")

```

Die Anzahl, wie oft die Simulation itertiert wurde beträgt 100000.

Die Anzahl, wie oft der Alarm ausgelöst wurde beträgt 235.

Die Anzahl, wie oft eingebrochen wurde beträgt 102.

Die Anzahl, wie oft ein Erdbeben stattfand beträgt 185.

Die Anzahl, wie oft John angerufen hat beträgt 5185.

Die Anzahl, wie oft Mary angerufen hat beträgt 1165.

Die Anzahl, wie oft beide angerufen haben beträgt 187.

Die Anzahl, wie oft beide angerufen haben, als ein Einbruch stattfand beträgt 135.

Die Anzahl, wie oft beide angerufen haben, als kein Einbruch stattfand beträgt 135.

Nachdem die Simulation abgelaufen ist, müssen nun die gefragten Wahrscheinlichkeiten aus der Aufgabenstellung berechnet werden.

Für die Wahrscheinlichkeiten, gilt für die Daten aus der Simulation, folgendes:

$$P(A) = \frac{\text{Alarme}}{\text{Iterationen}}$$

$$P(J) = \frac{\text{Anrufe John}}{\text{Iterationen}}$$

$$P(M) = \frac{\text{Anrufe Mary}}{\text{Iterationen}}$$

$$P(J \wedge M \mid \text{Ein}) = \frac{\text{Anrufe beide bei Einbruch}}{\text{Einbrüche}}$$

$$P(J \wedge M \mid \neg \text{Ein}) = \frac{\text{Anrufe beide kein Einbruch}}{\text{Iterationen} - \text{Einbrüche}}$$

$$P(J \wedge M) = \frac{\text{Anrufe beide}}{\text{Iterationen}}$$

$$P(\text{Ein} \mid J \wedge M) = \frac{\text{Anrufe beide bei Einbruch}}{\text{Anrufe beide}}$$

Diese werden am Ende dann noch in Prozenangaben ausgegeben.

```
[658]: probability_alarm : float = math.floor((count_alarm / iterations) * 10000) / 100
probability_john : float = math.floor((count_call_john / iterations) * 10000) /
↳ 100
probability_mary : float = math.floor((count_call_mary / iterations) * 10000) /
↳ 100
probability_call_both_if_bulgary : float = math.
↳ floor((count_call_both_if_bulgary / count_bulgary) * 10000) / 100
probability_call_both_if_no_bulgary : float = math.
↳ floor((count_call_both_if_no_bulgary / (iterations - count_bulgary)) *
↳ 10000) / 100
probability_call_both : float = math.floor((count_call_both / iterations) *
↳ 10000) / 100
probability_bulgary_if_both_call : float = math.
↳ floor((count_call_both_if_bulgary / count_call_both) * 10000) / 100

probability_cases : list = ["Wahrscheinlichkeit, dass ein Alarm ausgelöst
↳ wird", "Wahrscheinlichkeit, dass John anruft",
                             "Wahrscheinlichkeit, dass Mary anruft",
↳ "Wahrscheinlichkeit, dass beide Anrufen, wenn ein Einbruch stattfindet",
                             "Wahrscheinlichkeit, dass beide Anrufen, wenn kein
↳ Einbruch stattfindet", "Wahrscheinlichkeit, dass beide Anrufen",
```

```

        "Wahrscheinlichkeit, dass ein Einbruch stattfindet, wenn
        ↳ beide anrufen"]

probability_list : list = [probability_alarm, probability_john,
        ↳ probability_mary,
        probability_call_both_if_bulgary,
        ↳ probability_call_both_if_no_bulgary,
        probability_call_both, probability_bulgary_if_both_call]

for i in range(len(probability_list)):
    print(f"Die {probability_cases[i]} beträgt {probability_list[i]}%")

```

Die Wahrscheinlichkeit, dass ein Alarm ausgelöst wird beträgt 0.23%.

Die Wahrscheinlichkeit, dass John anruft beträgt 5.18%.

Die Wahrscheinlichkeit, dass Mary anruft beträgt 1.16%.

Die Wahrscheinlichkeit, dass beide Anrufen, wenn ein Einbruch stattfindet beträgt 50.98%.

Die Wahrscheinlichkeit, dass beide Anrufen, wenn kein Einbruch stattfindet beträgt 0.13%.

Die Wahrscheinlichkeit, dass beide Anrufen beträgt 0.18%.

Die Wahrscheinlichkeit, dass ein Einbruch stattfindet, wenn beide anrufen beträgt 27.8%.

Um die Unabhängigkeit für die Anrufe zu beurteilen, wird die folgende Äquivalenz verwendet. Dabei gilt, dass zwei Ereignisse voneinander unabhängig sind, wenn die Produktregel erfüllt ist, also gilt:

$$P(M \wedge J) = P(M) \cdot P(J)$$

Berechnen wir das für den Fall der Simulation gilt dann folgendes:

```

[659]: is_independent : bool
        tolerance = 0.01

        probability_mary_times_john = math.floor((probability_mary / 100) *
        ↳ (probability_john / 100) * 10000) / 10000

        if abs(probability_call_both - probability_mary_times_john) < tolerance:
            is_independent = True
            print("Die Anrufe von Mary und John sind unabhängig voneinander")

        else:
            is_independent = False
            print("Die Anrufe von Mary und John sind abhängig voneinander")

        print(f"Wert für \"Mary und John\" {probability_call_both}%")
        print(f"Wert für \"Mary mal John\" {probability_mary_times_john}%")

```


Die Anrufe von Mary und John sind abhängig voneinander
Wert für "Mary und John" 0.18%
Wert für "Mary mal John" 0.0006%

This notebook was converted with convert.ploomber.io